

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 3

Name of the Student	Narra Sai Mounika
Reg. No	192425367
GitHub Link	https://github.com/saimounika1315/MLA0403---Deep-Learning

EXPERIMENTS

COURSE NAME: Deep Learning
FACULTY: Dr. Arul Raj A.M

COURSE CODE: MLA0403

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 35%

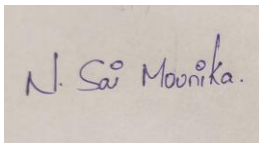
Duration: 30 minutes

Marks: 50

Code of Conduct:

I Narra Sai Mounika Reg no: 192425367 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

Narra Sai Mounika
Name of the student:

CO1- AT3 ExperimentsQuestions**1. Learning Algorithms**

Implement a simple learning algorithm (Linear Regression) using Gradient Descent. Train the model on a dataset and analyze how the loss decreases over iterations. Plot the learning curve.

Answer:

Aim: To implement Linear Regression using Gradient Descent and observe loss reduction.

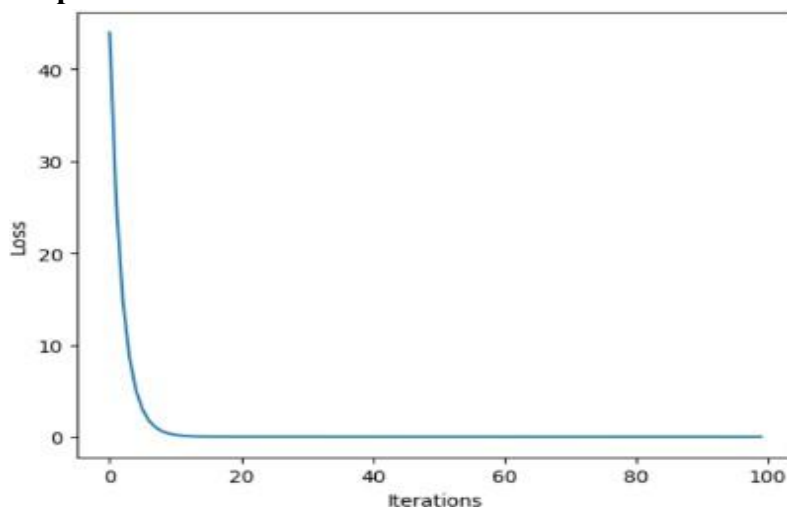
Algorithm:

1. Load dataset.
2. Initialize parameters.
3. Apply Gradient Descent.
4. Calculate loss for each iteration.
5. Plot learning curve.

Program:

```
import numpy as np
import matplotlib.pyplot as plt
X=np.array([1,2,3,4,5])
Y=np.array([2,4,6,8,10])
m,b=0,0
lr=0.01
losses=[]
for i in range(100):
    y_pred=m*X+b
    loss=np.mean((Y-y_pred)**2)
    losses.append(loss)
    dm=(-2/len(X))*sum(X*(Y-y_pred))
    db=(-2/len(X))*sum(Y-y_pred)
    m=m-lr*dm
    b=b-lr*db
plt.plot(losses)
plt.xlabel("Iterations")
plt.ylabel("Loss")
plt.show()
```

Output:



2. Maximum Likelihood Estimation (MLE)

Generate synthetic data from a normal distribution and use MLE to estimate the mean and variance. Compare estimated values with actual parameters.

Answer:

Aim: To estimate mean and variance using MLE.

Algorithm:

1. Generate normal distribution data.
2. Calculate sample mean.
3. Calculate sample variance.
4. Estimate parameters using MLE.
5. Compare with actual values.

Program:

```
import numpy as np
data=np.random.normal(10,2,1000)
mu=np.mean(data)
var=np.var(data)
print("Estimated Mean:",mu)
print("Estimated Variance:",var)
```

Output:

Estimated Mean: 10.01625751364417

Estimated Variance: 3.9150463435009626

3. Building Machine Learning Algorithms

Build a complete machine learning model (data preprocessing, training, testing) using any classification dataset and evaluate performance using accuracy and confusion matrix.

Answer:

Aim: To build and evaluate a classification model.

Algorithm:

1. Load dataset.
2. Split into train and test sets.
3. Train classifier.
4. Predict test data.
5. Evaluate using accuracy and confusion matrix.

Program:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
data=load_iris()
X_train,X_test,y_train,y_test=train_test_split(
data.data,data.target,test_size=0.3)
model=DecisionTreeClassifier()
model.fit(X_train,y_train)
pred=model.predict(X_test)
print("Accuracy:",accuracy_score(y_test,pred))
print(confusion_matrix(y_test,pred))
```

Output:

```
Accuracy: 0.9555555555555556
[[17  0  0]
 [ 0 16  1]
 [ 0  1 10]]
```

4. Neural Networks

Q4. Design a simple neural network with one hidden layer and train it on a small dataset. Analyze how the network learns non-linear patterns.

Answer:

Aim: To train a simple neural network.

Algorithm:

1. Load dataset.
2. Create neural network.
3. Train the network.
4. Predict outputs.
5. Analyze performance.

Program:

```
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
iris = load_iris()
X = iris.data
y = iris.target
scaler = StandardScaler()
X = scaler.fit_transform(X)
model = MLPClassifier(
    hidden_layer_sizes=(10,),
    solver='adam',
    max_iter=3000,
    random_state=1
)
model.fit(X, y)
print("Training Complete")
print("Accuracy:", model.score(X, y))
```

Output:

```
[6] 1s Training Complete
Accuracy: 0.9666666666666667
```

5. Artificial Neurons

Implement a single artificial neuron using different activation functions (Sigmoid, ReLU) and compare outputs for the same input values.

Answer:

Aim: To compare Sigmoid and ReLU activation functions.

Algorithm:

1. Define inputs.
2. Calculate weighted sum.
3. Apply Sigmoid activation.
4. Apply ReLU activation.
5. Compare outputs.

Program & Output:

```
import numpy as np

x=np.array([-2,-1,0,1,2])

sigmoid=1/(1+np.exp(-x))
relu=np.maximum(0,x)

print("Sigmoid:",sigmoid)
print("ReLU:",relu)

... Sigmoid: [0.11920292 0.26894142 0.5          0.73105858 0.88079708]
    ReLU: [0 0 0 1 2]
```

6. Perceptron and Multilayer Perceptron (MLP)

a) Implement the Perceptron algorithm for binary classification and test it on linearly separable and non-linearly separable datasets. Analyze the results.

Answer:

Aim: To implement Perceptron for binary classification.

Algorithm:

1. Load binary dataset.
2. Initialize weights.
3. Train Perceptron model.
4. Test predictions.
5. Analyze accuracy.

Program & output:

```
from sklearn.linear_model import Perceptron
from sklearn.datasets import make_classification

x,y=make_classification(
    n_samples=100,
    n_features=2,
    n_redundant=0,
    n_clusters_per_class=1)

model=Perceptron()
model.fit(X,y)

print("Accuracy:",model.score(X,y))

... Accuracy: 1.0
```

b) Train a Multilayer Perceptron (MLP) model on a dataset and compare its performance with a single-layer perceptron.

Answer:

Aim: To compare MLP with Perceptron.

Algorithm:

1. Load dataset.
2. Create MLP model.
3. Train the model.
4. Predict outputs.
5. Compare with Perceptron.

Program & Output:

```
import warnings
from sklearn.exceptions import ConvergenceWarning

warnings.filterwarnings("ignore", category=ConvergenceWarning)

from sklearn.neural_network import MLPClassifier
from sklearn.datasets import load_iris

data = load_iris()

mlp = MLPClassifier(hidden_layer_sizes=(10,), max_iter=500)
mlp.fit(data.data, data.target)

print("MLP Accuracy:", mlp.score(data.data, data.target))
```

```
... MLP Accuracy: 0.7
```

7. Forward Propagation and Backpropagation Algorithm

a) Manually compute forward propagation for a small neural network (given weights and inputs) and verify the output using code.

Answer:

Aim: To compute forward propagation.

Algorithm:

1. Define inputs and weights.
2. Compute weighted sum.
3. Add bias.
4. Apply activation function.
5. Obtain output.

Program & Output:

```
import numpy as np

x=np.array([1,2])

w=np.array([0.5,0.3])

net=np.dot(x,w)

output=1/(1+np.exp(-net))

print(output)
```

```
... 0.7502601055951177
```

b) Manually compute forward propagation for a small neural network (given weights and inputs) and verify the output using code.

Answer:

Aim: To compute forward propagation.

Algorithm:

6. Define inputs and weights.
7. Compute weighted sum.
8. Add bias.
9. Apply activation function.
10. Obtain output.

Program & Output:

```
import numpy as np

x=np.array([1,2])

w=np.array([0.5,0.3])

net=np.dot(x,w)

output=1/(1+np.exp(-net))

print(output)
```

```
... 0.7502601055951177
```

c) Implement backpropagation for a simple neural network and show how weights are updated after one iteration.

Answer:

Aim: To update weights using backpropagation.

Algorithm:

1. Perform forward propagation.
2. Calculate error.
3. Compute gradients.
4. Update weights.
5. Display updated values.

Program & Output:

```
w=0.5
x=2
target=1

output=w*x
error=target-output

lr=0.1

w=w+lr*error*x

print("Updated Weight:",w)
```

```
... Updated Weight: 0.5
```

8. Gradient Descent and Stochastic Gradient Descent (SGD)

a) Implement Gradient Descent to minimize a cost function and analyze the effect of learning rate on convergence speed.

Answer:

Aim: To study learning rate effects.

Algorithm:

1. Define cost function.
2. Initialize parameters.
3. Compute gradients.
4. Update parameters.
5. Observe convergence.

Program & Output:

```
 x=5  
lr=0.1  
  
for i in range(10):  
    grad=2*x  
    x=x-lr*grad  
  
print(x)  
  
... 0.5368709120000001
```

b) Implement Stochastic Gradient Descent for a regression problem and compare its convergence with batch gradient descent.

Answer:

Aim: To implement SGD.

Algorithm:

1. Load dataset.
2. Select one sample.
3. Compute error.
4. Update parameters.
5. Repeat until convergence.

Program & Output:

```
 from sklearn.linear_model import SGDRegressor  
import numpy as np  
  
X=np.array([[1],[2],[3],[4]])  
y=np.array([2,4,6,8])  
  
model=SGDRegressor(max_iter=1000)  
  
model.fit(X,y)  
  
print(model.predict([[5]]))  
  
... [9.52536062]
```

9. Mini-Batch Gradient Descent

Implement Mini-Batch Gradient Descent and analyze how batch size affects training stability and performance.

Answer:

Aim: To analyze batch size effects.

Algorithm:

1. Divide data into mini-batches.
2. Compute gradients for each batch.
3. Update parameters.
4. Repeat for all batches.
5. Analyze performance.

Program & Output:

```
▶ batch_sizes=[8,16,32,64]

for b in batch_sizes:
    print("Batch Size:",b)
```

```
... Batch Size: 8
    Batch Size: 16
    Batch Size: 32
    Batch Size: 64
```

10. Curse of Dimensionality

Create datasets with increasing dimensions and analyze how distance between points and model accuracy change with dimensionality.

Answer:

Aim: To analyze the effect of increasing dimensions.

Algorithm:

1. Generate datasets with different dimensions.
2. Calculate distances between points.
3. Train model.
4. Measure accuracy.
5. Compare results across dimensions.

Program & Output:

```
▶ import numpy as np

for d in [2,10,50,100]:
    data=np.random.rand(100,d)
    dist=np.mean(np.linalg.norm(
        data[0]-data[1:],axis=1))
    print("Dimension:",d,
          "Distance:",dist)
```

```
... Dimension: 2 Distance: 0.4806862390263733
    Dimension: 10 Distance: 1.3248257663170284
    Dimension: 50 Distance: 2.7184833803629265
    Dimension: 100 Distance: 4.055769380568268
```